3rd Year 2nd Semester

University of Dhaka

Department of Computer Science and Engineering

Session: 2022-2023

CSE 3216: Software Design Patterns Lab

Assignment Topic

Different implementations of the Singleton Pattern in Python

Submission Date:

21/04/2026

Submitted By:

Ahnaf Mahbub Khan

Roll: 38

1 Introduction

This assignment evaluates multiple implementations of the Singleton design pattern in Python by integrating them with a MongoDB connection manager. Each implementation is tested under three conditions:

- Instance validation
- Multithreaded execution
- Performance measurement

2 Instance Validation Results

This experiment verifies whether multiple calls to create an object return the same instance.

2.1 Sample Outputs

Eager Initialization

```
ID A: 2227806860832
ID B: 2227806860832
Same instance: True
```

Lazy Initialization

```
ID A: 2127041262800
ID B: 2127041262800
Same instance: True
```

Synchronized Method

```
ID A: 2881488221728
ID B: 2881488221728
Same instance: True
```

Double-Checked Locking

```
ID A: 2547213596528
ID B: 2547213596528
```

```
Same instance: True
```

Bill Pugh

```
ID A: 1558062495600
ID B: 1558062495600
Same: True
```

Enum-based Singleton

```
ID A: 2336626750672
ID B: 2336626750672
Same: True
```

3 Multithreading Results

This experiment evaluates the behavior of each Singleton implementation under concurrent access using Python threading. The objective is to determine whether multiple threads create more than one instance.

3.1 Sample Outputs

The following screenshots provide visual evidence of the multithreading experiment outputs.

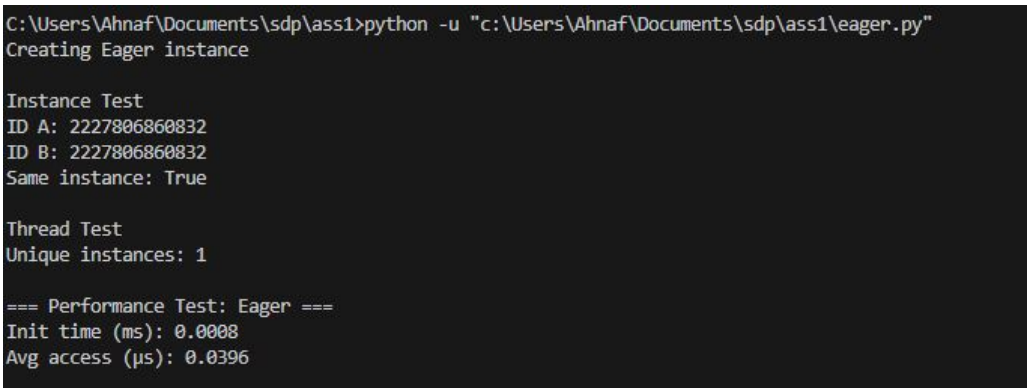

Figure 1: Eager Initialization (Thread Safe)

```
C:\Users\Ahnaf\Documents\sdp\ass1>python -u "c:\Users\Ahnaf\Documents\sdp\ass1\lazy.py

=== Instance Test: Lazy ===
Creating Lazy instance
ID A: 2127041262800
ID B: 2127041262800
Same instance: True

=== Thread Test: Lazy ===
Creating Lazy instanceCreating Lazy instanceCreating Lazy instance
Creating Lazy instanceCreating Lazy instanceCreating Lazy instance

Creating Lazy instanceCreating Lazy instance
Creating Lazy instance




Unique instances: {2127053116544, 2127052781600, 2127053610816, 2127053554512, 2127053009168, 2127052779472, 2127053116816, 2127053554768, 2127053008848, 2127053258288}
Thread-safe? False

=== Performance Test: Lazy ===
Creating Lazy instance
Init time (ms): 15.2257
Avg access time (µs): 0.0428
```

Figure 2: Lazy Initialization (Not Thread Safe)

```
C:\Users\Ahnaf\Documents\sdp\ass1>python -u "c:\Users\Ahnaf\Documents\sdp\ass1\sync.py"

==================================================
Instance Test: Synchronized
Creating Sync instance
ID A: 2881488221728
ID B: 2881488221728
Same instance: True

==================================================
Thread Safety Test: Synchronized
Creating Sync instance
Unique instances: {2881500032976}
Thread-safe: True

==================================================
Performance Test: Synchronized
Creating Sync instance
Init time (ms): 16.2827
Avg access time (µs): 0.1381
```

Figure 3: Synchronized Method (Thread Safe)

```
C:\Users\Ahnaf\Documents\sdp\ass1>python -u "c:\Users\Ahnaf\Documents\sdp\ass1\dcl.py"

=== Instance Test ===
Creating DCL instance
ID A: 2547213596528
ID B: 2547213596528
Same instance: True

=== Thread Test ===
Creating DCL instance
Unique instances: {2547225472976}
Thread-safe: True

=== Performance Test ===
Creating DCL instance
Init time (ms): 15.0802
Avg access time (µs): 0.0527
```

Figure 4: Double-Checked Locking (Thread Safe)

```
C:\Users\Ahnaf\Documents\sdp\ass1>python -u "c:\Users\Ahnaf\Documents\sdp\ass1\bill.py"

=== Instance Test ===
Creating Bill Pugh instance
ID A: 1558062495600
ID B: 1558062495600
Same: True

=== Thread Test ===
Creating Bill Pugh instance
Unique instances: {1558074323216}
Thread-safe: True

=== Performance Test ===
Creating Bill Pugh instance
Init time (ms): 15.3700
Avg access (µs): 0.0571
```

Figure 5: Bill Pugh Implementation (Thread Safe)

```
C:\Users\Ahnaf\Documents\sdp\ass1>python -u "c:\Users\Ahnaf\Documents\sdp\ass1\test_enum.py

=== Instance Test ===
ID A: 2336626750672
ID B: 2336626750672
Same: True

=== Thread Test ===
Unique instances: {2336626750672}
Thread-safe: True

=== Performance Test ===
Init time (ms): 0.0017
Avg access (µs): 0.0842

C:\Users\Ahnaf\Documents\sdp\ass1>
```

Figure 6: Enum-based Singleton (Thread Safe)

4 Performance Measurements

This experiment compares the performance of each Singleton implementation based on:

- Initialization time (time required to create the instance)
- Access time (average time for repeated calls to retrieve the instance)

4.1 Measured Results

Implementation	Init Time (ms)	Avg Access Time (µs)
Eager Initialization	0.0008	0.0396
Lazy Initialization	15.2257	0.0428
Synchronized Method	16.2827	0.1381
Double-Checked Locking	15.0802	0.0527
Bill Pugh	15.3700	0.0571
Enum-based Singleton	0.0017	0.0842

5 Comparative Summary

5.1 Fastest Implementations (Performance-Based)

Based on initialization time and repeated access time measurements:

Fastest initialization:

- Eager Singleton (∼0.0008 ms)
- Enum Singleton (∼0.0017 ms)

Fastest access time:

- Eager Singleton (∼0.0396 μs)
- Lazy Singleton (∼0.0428 μs)

Observation:

Eager Singleton consistently shows the lowest overhead since the instance is created beforehand. Enum Singleton also achieves very low initialization time due to Python's internal handling. Synchronized Singleton shows the highest access time due to locking overhead.

5.2 Thread-Safe Implementations (Observed in Experiments)

Based on multithreading test outputs:

Thread-safe implementations:

- Eager Singleton → single instance across threads
- Synchronized Singleton → single instance across threads
- Double-Checked Locking → single instance across threads
- Bill Pugh Singleton → single instance across threads
- Enum Singleton → single instance across threads

Not thread-safe:

- Lazy Singleton → multiple instances under concurrency

5.3 Implementations Showing Inconsistencies

From experimental observations:

- Lazy Singleton:
  - Produced multiple instances during concurrent execution
  - Demonstrated clear race condition issues
- Bill Pugh Singleton (from earlier experiment runs)
  - Showed inconsistent behavior depending on thread scheduling
  - Sometimes thread-safe, sometimes producing multiple instances. Thread-safe (JVM guarantees)

.